G2_Task

Hello,

Please read the following carefully:

As a Software Engineering research team at the Polytechnique Montréal (Canada), we are interested in understanding developers' assessment of coding task difficulty. You will be provided with a series of 12 coding questions. Questions are divided into two parts:

- First, you will be given three instructions representing coding tasks and you will be asked to write, in a few words, what is the underlying task represented by all these instructions. The instructions represent the same tasks, but they vary in wording and information they give. In a nutshell, you have to distill the essence of the task and write it in a few words.

- In a second time, you will be asked to assess how difficult implementing this ***task*** would be. Note we ask the assessment based on the task you will have worded in the previous part, not based on any of the instructions presented earlier. First, you will be asked your subjective judgment on a scale from Very Easy to Very Difficult. Then, to make judging the difficulty more relatable, we adopt a similar approach as what is used in agile development and planning poker, i.e. you will be asked to evaluate how long it would take to implement a task (in minutes here). Consider in your assessment the knowledge needed to implement the task, the external information you would need to use, whether you would need the assistance of a LLM such as ChatGPT (and so the time it would take to debug the obtained code) etc. Finally, you will have to explain in a few words why you gave such difficulty ratings.

To give you an idea of the range of tasks you will need to assess, the first section of this questionnaire will be two examples we devised to illustrate the questionnaire.

Completing the task will take about 30 minutes.

Note also that all examples are written using Python 3.10 syntax.

Please make sure to answer all the questions to the best of your ability.

Please note that your identity and personal data will not be disclosed, while we plan to use the aggregated results and anonymized responses as part of a scientific publication. If you have any questions about the questionnaire or our research, please do not hesitate to contact us.

By proceeding, you consent to take part in the study under the conditions described previously.

Thank you,

Florian Tambon (florian-2.tambon@polymtl.ca)
Cyrine Zid (cyrine.zid@polymtl.ca)
Amin Nikanjam (amin.nikanjam@polymtl.ca)
Foutse Khomh (foutse.khomh@polymtl.ca)
Giuliano Antoniol (giuliano.antoniol@polymtl.ca)

---

* Indique une question obligatoire

1. **Please enter your prolific ID :** *

Example of tasks

As we ask of you some estimation of difficulty and the tasks dealt in this questionnaire can be a bit different from what you can be used to as a developer, we give you examples of some tasks along with some assessments.

Note that this assessment is **our assessment**. You may have a different opinion based on your background. The idea of those examples is to give an extreme range of what type of tasks you can expect in the questionnaire.

As a reminder: You will first be given three instructions and asked to explain what is the task is about in a few words. Then, you will have to rate the difficulty of the ***task*** you just described (not accounting for particular instruction). Rating the difficulty is done in two steps: subjective difficulty and approximate time needed. Finally, you will be asked to say why you gave such rating.

Example 1

Instruction 1

```
def subtract(c1, c2):
        """
        Deliver the result of subtracting two complex numbers,
        specified as 'c1' and 'c2', as a complex output.
        :param c1: The first complex number,complex.
        :param c2: The second complex number,complex.
        :return: The difference of the two complex numbers,complex.
        """
```

Instruction 2

```
def subtract(c1, c2):
        """
        The method receives two complex numbers 'c1' and 'c2', from which it
        subtracts 'c2.real' from 'c1.real' to determine the real portion, and
        subtracts 'c2.imag' from 'c1.imag' to determine the imaginary portion.
        It constructs and returns a new complex number using these results
        with the 'complex()' function.
        :param c1: The first complex number,complex.
        :param c2: The second complex number,complex.
        :return: The difference of the two complex numbers,complex.
        """
```

Instruction 3

```
def subtract(c1, c2):
        """
        Subtracts two complex numbers.
        :param c1: The first complex number,complex.
        :param c2: The second complex number,complex.
        :return: The difference of the two complex numbers,complex.
        >>> complexCalculator = ComplexCalculator()
        >>> complexCalculator.subtract(1+2j, 3+4j)
        (-2-2j)
        """
```

Answer

The task encapsulated by the **subtract**

function

is to compute the subtraction between two complex numbers.

This task has a subjective difficulty of **Very Easy** as assessed by us and would take about **1 min**
to implement. There is no particular difficulty, the task is very straightforward without any corner case, it just requires to know what a complex is, and the standard Python library for it.

Example 2

Context

```python
import numpy as np
class MetricsCalculator2:
    """
    The class provides to calculate Mean Reciprocal Rank (MRR) and Mean Average Precision (MAP)
    based on input data, where MRR measures the ranking quality and MAP measures the average precision.
    """

    def __init__(self):
        pass

    @staticmethod
    def map(data):
        pass
```

Instruction 1

```python
def mrr(data):
        """
        Compute the MRR of the input data. MRR is a widely used evaluation index.
        It is the mean of reciprocal rank.
        :param data: the data must be a tuple, list 0,1,eg.([1,0,...],5).
        In each tuple (actual result,ground truth num),ground truth num is the total ground num.
        ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
        1 stands for a correct answer, 0 stands for a wrong answer.
        :return: if input data is list, return the recall of this list. if the input data is list of list, return the
        average recall on all list. The second return value is a list of precision for each input.
        >>> MetricsCalculator2.mrr(([1, 0, 1, 0], 4))
        >>> MetricsCalculator2.mrr(([([1, 0, 1, 0], 4), ([0, 1, 0, 1], 4)])
        1.0, [1.0]
        0.75, [1.0, 0.5]

        """
```

Instruction 2

```python
def mrr(data):
        """
        Assess the Mean Reciprocal Rank (MRR) from the input 'data'.
        MRR assesses the average of reciprocal ranks of outcomes.
        The input structure 'data' must be a tuple consisting of a list of
        binary values and its total count or a list of such tuples.
        Each binary value (0 or 1) represents the correctness of an answer.
        If 'data' is a tuple, the function returns its mean reciprocal rank.
        If 'data' is a list, the function returns the overall average MRR along
        with a list showing reciprocal ranks for each component tuple.

        :param data: the data must be a tuple, list 0,1,eg.([1,0,...],5).
        In each tuple (actual result,ground truth num),ground truth num is the total ground num.
        ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
        1 stands for a correct answer, 0 stands for a wrong answer.
        :return: if input data is list, return the recall of this list. if the input data is list of list, return the
        average recall on all list. The second return value is a list of precision for each input.

        """
```

Instruction 3

```
def mrr(data):
        """
        Evaluate the Mean Reciprocal Rank (MRR) from 'data'. This metric calculates the mean of reciprocal
        ranks for correct answers. Input 'data' could be either a tuple containing binary correctness indicators
        and the aggregate count of answers, or multiple tuples of this type in a list. For each tuple, the function
        measures reciprocal ranks by multiplying the binary indicators by reciprocals of their positions, finding the
        earliest nonzero value to establish the rank of the first correct response. If the input 'data' is a tuple,
        the MRR calculated for this specific tuple is returned; if it's a list, the function summarizes by providing
        both the average MRR and a list of individual MRR values for each tuple.

        :param data: the data must be a tuple, list 0,1,eg.([1,0,...],5).
        In each tuple (actual result,ground truth num),ground truth num is the total ground num.
        ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
        1 stands for a correct answer, 0 stands for a wrong answer.
        :return: if input data is list, return the recall of this list. if the input data is list of list, return the
        average recall on all list. The second return value is a list of precision for each input.

        """
```

Answer

The task encapsulated by the **mrr** function is to compute the
Mean Reciprocal Rank of binary tuples data.

This task has a subjective difficulty of **Very Hard** as assessed by us and would take about **40 min**
to implement. The difficulty mainly branches from dealing with the
tuple data and how the formula is processed accordingly (with a few corner cases). This can take
additional time if the Mean Reciprocal Rank is not known.

Question 1:

Below are instructions for implementing the function.

Instruction 1

```
from typing import List

def all_prefixes(string: str) -> List[str]:
    """
    Define a function 'all_prefixes' that accepts a string argument.
    This function intends to build and return a catalog of every prefix of the string,
    sorted by ascending length. By iterating through the string from beginning to ending,
    it extracts a substring from the start up to the current position in each loop and adds
    these to a list. The function finally returns this collection of substrings,
    each representing a prefix of the string, increasing in length.
    """
```

Instruction 2

```python
from typing import List

def all_prefixes(string: str) -> List[str]:
    """ Return list of all prefixes from shortest to longest of the input string
    >>> all_prefixes('abc')
    ['a', 'ab', 'abc']
    """
```

Instruction 3

```python
from typing import List

def all_prefixes(string: str) -> List[str]:
    """
    Construct a function named 'all_prefixes' that takes a single string as input.
    The function is designed to generate a list that includes all initial segments
    of the input string, arranged from the shortest prefix to the longest.
    It accomplishes this by iterating over the string from the first character to the last,
    on each iteration, slicing the string from the beginning to the current index and gathering
    these segments in a list. This list, which represents all possible prefixes in ascending
    order of size, is then returned.
    """
```

2. In a few words, what is the task this function aims to implement? *

_____

3. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

4. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ◯ 1 min
- ◯ 5 min
- ◯ 10 min
- ◯ 20 min
- ◯ 40 min
- ◯ > 40 min

5. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 2:

Below are instructions for implementing the function:

Context

```
class RPGCharacter:
    """
    The class represents a role-playing game character, which allows to attack other characters,
    heal, gain experience, level up, and check if the character is alive.
    """

    def __init__(self, name, hp, attack_power, defense, level=1):
        """
        Initialize an RPG character object.
        :param name: strm, the name of the character.
        :param hp: int, The health points of the character.
        :param attack_power: int, the attack power of the character.
        :param defense: int, the defense points of the character.
        :param level: int, the level of the character. Default is 1.
        """
        self.name = name
        self.hp = hp
        self.attack_power = attack_power
        self.defense = defense
        self.level = level
        self.exp = 0

    def attack(self, other_character):
        pass

    def heal(self):
        pass

    def gain_exp(self, amount):
        pass

    def level_up(self):
        pass
```

Instruction 1

```
def is_alive(self):
  """
  Verify whether the player is alive by examining their health points.
  Should 'self.hp' be greater than 0, return 'True', otherwise 'False'.
  :return: True if the hp is larger than 0, or False otherwise.
  """
```

Instruction 2

```
def is_alive(self):
  """
  Check if player is alive.
  :return: True if the hp is larger than 0, or False otherwise.
  >>> player_1 = RPGCharacter('player 1', 100, 10, 3)
  >>> player_1.is_alive()
  True
  """
```

Instruction 3

```
def is_alive(self):
  """
  Establish if the player is alive by inspecting their health points.
  Return 'True' if 'self.hp' is more than 0, else return 'False'.
  :return: True if the hp is larger than 0, or False otherwise.
  """
```

6. In a few words, what is the task this function aims to implement? *

_____

7. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

8. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- ◯ 1 min
- ◯ 5 min
- ◯ 10 min
- ◯ 20 min
- ◯ 40 min
- ◯ > 40 min

9. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 3:

Below are instructions for implementing the function:

Instruction 1

```python
from typing import List


def parse_music(music_string: str) -> List[int]:
    """
    Input to this function is a string representing musical notes
    in a special ASCII format.
    Your task is to parse this string and return list of integers
    corresponding to how many beats does each not last.

    Here is a legend:
    'o' - whole note, lasts four beats
    'o|' - half note, lasts two beats
    '.|' - quater note, lasts one beat

    >>> parse_music('o o| .| o| o| .| .| .| .| o o')
    [4, 2, 1, 2, 2, 1, 1, 1, 1, 4, 4]
    """
```

Instruction 2

```python
from typing import List


def parse_music(music_string: str) -> List[int]:
    """
    Develop a function called 'parse_music' that receives a string parameter
    illustrating musical notes in a unique ASCII representation, designed
    to generate a list of integers each representing the beat duration
    of a note. Inside this function, a nested function calculates the beat
    length for each note type, where a 'o' implies a whole note with four beats,
    a'o|' symbolizes a half note with two beats, and a '.|' indicates a quarter note
    of one beat. An empty string input leads to an output of an empty list.
    The function overall breaks down the input string into individual notes,
    assigns them beat values using the nested function, and outputs a list of these values.
    """
```

Instruction 3

```python
from typing import List


def parse_music(music_string: str) -> List[int]:
    """
    Write a function named "parse_music" which takes a string
    input "music_string" representing musical notes in a special
    ASCII format, and returns a list of integers representing the
    duration of each note in beats. Within the function, an inner
    function "count_beats" is used to determine the duration in beats
    of individual notes: the pattern "o" represents a whole note
    lasting four beats, the pattern "o|" represents a half note
    lasting two beats, and the pattern ".|" represents a quarter note
    lasting one beat. If the input string "music_string" is empty,
    the function returns an empty list. Otherwise, the main body of
    the function splits the input string into individual note
    representations using "split()", and then uses "map()" to apply
    "count_beats" across these representations, converting them into
    their corresponding beat durations. Finally, the function returns
    this list of durations.
    """
```

10. In a few words, what is the task this function aims to implement? *

11. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

12. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

13. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 4:

Below are instructions for implementing the function:

Context

```
import sqlite3
class StudentDatabaseProcessor:
    """
    This is a class with database operation, including inserting student information,
    searching for student information by name, and deleting student information by name.
    """

    def __init__(self, database_name):
        """
        Initializes the StudentDatabaseProcessor object with the specified database name.
        :param database_name: str, the name of the SQLite database.
        """
        self.database_name = database_name

    def create_student_table(self):
        pass

    def insert_student(self, student_data):
        pass

    def search_student_by_name(self, name):
        pass
```

Instruction 1

```
def delete_student_by_name(self, name):
  """
  By specifying a student's name, this function deletes
  the corresponding record from the 'students' table.
  The process begins when the function connects to the
  designated database with 'sqlite3.connect',
  using 'self.database_name' as the target database.
  It then sets up a cursor to execute queries and crafts a
  deletion SQL query in 'delete_query', targeting records in
  the 'students' table where 'name' equals the passed 'name'.
  The function carries out this query via 'cursor.execute(delete_query, (name,))',
  commits the changes with 'conn.commit()', and concludes by shutting down
  the connection using 'conn.close()'.
  :param name: str, the name of the student to delete.
  :return: None
  """
```

Instruction 2

```
def delete_student_by_name(self, name):
  """
  Deletes a student from the "students" table by their name.
  :param name: str, the name of the student to delete.
  :return: None
  >>> processor = StudentDatabaseProcessor("students.db")
  >>> processor.create_student_table()
  >>> student_data = {'name': 'John', 'age': 15, 'gender': 'Male', 'grade': 9}
  >>> processor.insert_student(student_data)
  >>> processor.delete_student_by_name("John")
  """
```

Instruction 3

```python
def delete_student_by_name(self, name):
    """
    This function removes a student from a database's 'students'
    table using their name. Initially, it connects to the database
    with the 'database_name' class attribute. It crafts a SQL deletion
    query that targets a student based on the supplied name string parameter.
    After running the deletion query, the function commits the transaction
    to save the modifications and shuts the database connection.
    :param name: str, the name of the student to delete.
    :return: None
    """
```

14. In a few words, what is the task this function aims to implement? *

_____

15. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

16. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

17. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 5:

Below are instructions for implementing the function:

Instruction 1

```
def get_max_triples(n):
    """
    Create a function called 'get_max_triples' that accepts a positive
    integer 'n' as an argument, creates a list 'a' with each entry 'a[i]'
    defined as 'i * i - i + 1'. The function should determine and return
    how many triples '(a[i], a[j], a[k])' can be formed such that 'i < j < k'
    and the sum of the elements in the triple is divisible by 3. This is
    achieved by using nested loops for combination checking and modulo
    operation to filter triples.
    """
```

Instruction 2

```
def get_max_triples(n):
    """
    You are given a positive integer n. You have to create an integer array a of length n.
        For each i (1 ≤ i ≤ n), the value of a[i] = i * i - i + 1.
        Return the number of triples (a[i], a[j], a[k]) of a where i < j < k,
    and a[i] + a[j] + a[k] is a multiple of 3.

    Example :
        Input: n = 5
        Output: 1
        Explanation:
        a = [1, 3, 7, 13, 21]
        The only valid triple is (1, 7, 13).
    """
```

Instruction 3

```
def get_max_triples(n):
    """
    Write a function 'get_max_triples', which is given a positive integer 'n',
    generates an array 'a' per the formula 'a[i] = i * i - i + 1',
    and then determines the number of triples '(a[i], a[j], a[k])'
    where 'i < j < k' and their sum tests true for division by 3 using modulo.
    It returns this count, calculated through cross-examining potential
    triples in nested iteration process.
    """
```

18. In a few words, what is the task this function aims to implement? *

_____

19. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

20. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

21. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 6:

Below are instructions for implementing the function:

Context

```
class NumericEntityUnescaper:
    """
    This is a class that provides functionality to replace numeric entities
    with their corresponding characters in a given string.
    """

    def __init__(self):
        pass

    @staticmethod
    def is_hex_char(char):
        pass



```

Instruction 1

```
def replace(self, string):
        """
        Substitute numeric character references in the given 'string'
        with the corresponding Unicode characters they represent.
        :param string: str, the input string containing numeric
        character references.
        :return: str, the input string with numeric character references
                replaced with their corresponding Unicode characters.

        """
```

Instruction 2

```
def replace(self, string):
        """
        Replaces numeric character references (HTML entities)
        in the input string with their corresponding Unicode characters.
        :param string: str, the input string containing numeric character references.
        :return: str, the input string with numeric character references replaced with
                their corresponding Unicode characters.
        >>> unescaper = NumericEntityUnescaper()
        >>> unescaper.replace("&#65;&#66;&#67;")
        'ABC'
        """
```

Instruction 3

```
def replace(self, string):
        """
        Replace numeric character references in the input string "string"
        with their corresponding Unicode characters. The function scans through
        the string, identifying sequences that start with "&#" which indicate
        the beginning of a numeric entity. It determines whether the entity is
        in hexadecimal format. It then reads the numeric value until it sees
        a semicolon, which marks the end of the entity. This numeric value
        is converted into a Unicode character, which replaces the original entity
        in the output string.
        :param string: str, the input string containing numeric character references.
        :return: str, the input string with numeric character references replaced
                with their corresponding Unicode characters.
        """
```

22. In a few words, what is the task this function aims to implement? *

_____

23. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

24. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

25. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 7:

Below are instructions for implementing the function:

Instruction 1

```
def common(l1: list, l2: list):
    """
    Write a function named 'common' which takes two lists as input.
    The function aims to identify and return a sorted list of unique
    elements that are common to both input lists. It first converts each
    list into a set to remove duplicate elements and then finds
    the intersection of these sets to determine the common elements.
    Finally, it converts the resulting set of common elements into a
    list and returns it sorted.
    """
```

Instruction 2

```
def common(l1: list, l2: list):
    """
    Return sorted unique common elements for two lists.
    >>> common([1, 4, 3, 34, 653, 2, 5], [5, 7, 1, 5, 9, 653, 121])
    [1, 5, 653]
    >>> common([5, 3, 2, 8], [3, 2])
    [2, 3]
    """
```

Instruction 3

```
def common(l1: list, l2: list):
    """
    Develop a function 'common' that generates a list,
    sorted and containing only unique items,
    found in both supplied lists.

    """
```

26. In a few words, what is the task this function aims to implement? *

_____

27. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

28. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

29. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 8:

Below are instructions for implementing the function:

Context

```
class AvgPartition:
    """
    This is a class that partitions the given list into different blocks by
    specifying the number of partitions, with each block having a uniformly
    distributed length.
    """

    def __init__(self, lst, limit):
        """
        Initialize the class with the given list and the number of partitions,
        and check if the number of partitions is greater than 0.
        """
        self.lst = lst
        self.limit = limit

    def get(self, index):
        pass
```

Instruction 1

```
def setNum(self):
  """
  Assess the size of each partition and the remainder after dividing a list.
  It should return these measurements in a tuple: partition size and
  leftover elements.
  :return: the size of each block and the remainder of the division, tuple.
  """
```

Instruction 2

```
def setNum(self):
  """
  Calculate the size of each block and the remainder of the division.
  :return: the size of each block and the remainder of the division, tuple.
  >>> a = AvgPartition([1, 2, 3, 4], 2)
  >>> a.setNum()
  (2, 0)
  """
```

Instruction 3

```
def setNum(self):
  """
  Compute the size of each partition block and the leftover
  count after division when splitting a list. The procedure begins
  by calculating the full count of components in the list ('len(self.lst)')
  and divides this by the intended partition count ('self.limit')
  to determine each partition's size ('size'). Next, the function finds the
  remainder ('remainder') using the modulus of the total count ('len(self.lst)')
  with 'self.limit'. It returns these calculated values as a tuple; the first
  part represents the size of each partition ('size') and the second part the
  leftover count ('remainder').
  :return: the size of each block and the remainder of the division, tuple.
  """
```

30. In a few words, what is the task this function aims to implement? *

31. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

32. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

33. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 9:

Below are instructions for implementing the function:

Instruction 1

```
def special_factorial(n):
    """
    Define a function labelled 'special_factorial' which takes a single
    integer parameter 'n' and produces the Brazilian factorial of 'n'.
    Firstly, the variables 'fac' and 'ans' are set to 1, with a loop
    following from 2 to 'n'. In each loop iteration, 'fac' is multiplied by 'i',
    calculating the factorial at 'i'. Consequentially, 'ans' multiplies itself by 'fac',
    integrating the factorial products. The definitive result, 'ans', is returned,
    which is the aggregate of all the consecutive factorial products until 'n'.
    """
```

Instruction 2

```
def special_factorial(n):
    """The Brazilian factorial is defined as:
    brazilian_factorial(n) = n! * (n-1)! * (n-2)! * ... * 1!
    where n > 0

    For example:
    >>> special_factorial(4)
    288

    The function will receive an integer as input and should return the special
    factorial of this integer.
    """
```

Instruction 3

```
def special_factorial(n):
    """
    Develop a function 'special_factorial' to evaluate
    the special factorial for a specific integer.
    This special factorial is essentially the multiplication
    of each factorial from 1 to the specified integer.
    """
```

34. In a few words, what is the task this function aims to implement? *

35. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

36. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

37. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 10:

Below are instructions for implementing the function:

Context

```python
class HRManagementSystem:
    """
    This is a class as personnel management system that implements functions such as adding, deleting,
    querying, and updating employees
    """

    def __init__(self):
        """
        Initialize the HRManagementSystem withan attribute employees, which is an empty dictionary.
        """
        self.employees = {}

    def add_employee(self, employee_id, name, position, department, salary):
        pass

    def remove_employee(self, employee_id):
        pass

    def update_employee(self, employee_id: int, employee_info: dict):
        pass

    def list_employees(self):
        pass
```

Instruction 1

```python
def get_employee(self, employee_id):
        """
        Seek an employee's details in the HRManagementSystem by
        making use of 'employee_id'.
        If this employee is present in the system, deliver
        their information; otherwise, issue 'False'.
        :param employee_id: The employee's id, int.
        :return: If the employee is already in the HRManagementSystem,
        returns the employee's information, otherwise,
                returns False.
        """
```

Instruction 2

```python
def get_employee(self, employee_id):
        """
        Get an employee's information from the HRManagementSystem.
        :param employee_id: The employee's id, int.
        :return: If the employee is already in the HRManagementSystem,
        returns the employee's information, otherwise, returns False.
        >>> hrManagementSystem = HRManagementSystem()
        >>> hrManagementSystem.employees = {1: {'name': 'John', 'position': 'Manager',
                'department': 'Sales', 'salary': 100000}}
        >>> hrManagementSystem.get_employee(1)
        {'name': 'John', 'position': 'Manager', 'department': 'Sales', 'salary': 100000}
        >>> hrManagementSystem.get_employee(2)
        False
        """
```

Instruction 3

```
def get_employee(self, employee_id):
        """
        Query the HRManagementSystem for an employee's
        information using 'employee_id' as the key.
        Should the employee be recorded in the system,
        return their information; if not, return 'False'.
        :param employee_id: The employee's id, int.
        :return: If the employee is already in the
        HRManagementSystem, returns the employee's information,
        otherwise, returns False.
        """
```

38. In a few words, what is the task this function aims to implement? *

_____

39. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- ◯ Very Easy
- ◯ Easy
- ◯ Not Easy Nor Difficult
- ◯ Difficult
- ◯ Very Difficult

40. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

41. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 11:

Below are instructions for implementing the function

Instruction 1

```
def prime_fib(n: int):
    """
    Write a function `prime_fib` with an integer parameter `n`
    that yields the n-th prime number in the Fibonacci sequence.
    Use the 'math' module for any necessary arithmetic operations
    and define an inner function called `is_prime(p)` for primality
    testing. This tests if `p` at less than 2 should return False,
    or through a loop from 2 to the square root of `p` + 1,
    checks if `p` has divisors in that range, returning False if it has,
    otherwise True. Begin with `a` and `b` as 0 and 1 in Fibonacci numbers
    and a `c_prime` counter at 0. Iterate in a while loop till `c_prime`
    hasn't reached `n`, increase Fibonacci terms `a` and `b`, test if `b`
    is prime, and increment `c_prime` upon a prime `b`. Eventually, give
    back `b` when loop exits.
    """
```

Instruction 2

```
def prime_fib(n: int):
    """
    prime_fib returns n-th number that is a Fibonacci number and it's also prime.
    >>> prime_fib(1)
    2
    >>> prime_fib(2)
    3
    >>> prime_fib(3)
    5
    >>> prime_fib(4)
    13
    >>> prime_fib(5)
    89
    """
```

Instruction 3

```
def prime_fib(n: int):
    """
    Frame a function `prime_fib` taking an integer `n`
    to identify and return the n-th Fibonacci number which
    is prime. It should harness the power of the 'math' module
    for mathematical calculations. This function houses an
    internal function `is_prime(p)` for checking if `p`,
    less than 2 will promptly return False or cycles through
    numbers from 2 to the increment of the integer square root of `p`.
    If it finds divisibility, it gives out False; otherwise True.
    Starting variables `a` and `b` of the Fibonacci series at 0 and 1,
    and a count `c_prime` at 0, it runs a loop until `c_prime` attains `n`.
    It updates Fibonacci numbers and checks the primality of `b`,
    increasing `c_prime` when `b` is prime. Ultimately, it returns `b`.
    """
```

42. In a few words, what is the task this function aims to implement? *

43. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

44. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

45. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 12:

Below are instructions for implementing the function:

Context

```
class ArgumentParser:
    """
    This is a class for parsing command line arguments to a dictionary.
    """

    def __init__(self):
        """
        Initialize the fields.
        self.arguments is a dict that stores the args in a command line
        self.requried is a set that stores the required arguments
        self.types is a dict that stores type of every arguments.
        >>> parser.arguments
        {'key1': 'value1', 'option1': True}
        >>> parser.required
        {'arg1'}
        >>> parser.types
        {'arg1': 'type1'}
        """
        self.arguments = {}
        self.required = set()
        self.types = {}

    def get_argument(self, key):
        pass

    def add_argument(self, arg, required=False, arg_type=str):
        pass

    def _convert_type(self, arg, value):
        pass
```

Instruction 1

```
def parse_arguments(self, command_string):
        """
        Interpret the provided 'command_string' into its constituent arguments,
        checking for the presence of all imperative arguments.Employ the '_convert_type'
        function to assert that every argument is maintained with its exact type
        within the 'arguments' dictionary. Output a tuple, where the initial
        element is 'True' if all necessary arguments are gathered, else 'False',
        and the subsequent element is 'None' if no arguments are omitted, otherwise
        present a set of missing argument identifiers.
        :param command_string: str, command line argument string, formatted like
                "python script.py --arg1=value1 -arg2 value2 --option1 -option2"
        :return tuple: (True, None) if parsing is successful, (False, missing_args)
                if parsing fails,

        """
```

Instruction 2

```
def parse_arguments(self, command_string):
        """
        Parses the given command line argument string and
        invoke _convert_type to stores the parsed result in
        specific type in the arguments dictionary.
        Checks for missing required arguments, if any, and returns
        False with the missing argument names, otherwise returns True.
        :param command_string: str, command line argument string,
        formatted like "python script.py --arg1=value1 -arg2 value2 --option1 -option2"
        :return tuple: (True, None) if parsing is successful,
        (False, missing_args) if parsing fails,
        where missing_args is a set of the missing argument names which are str.
        >>> parser.parse_arguments("python script.py --arg1=value1 -arg2 value2 --option1 -option2")
        (True, None)
        >>> parser.arguments
        {'arg1': 'value1', 'arg2': 'value2', 'option1': True, 'option2': True}

        """
```

Instruction 3

```
def parse_arguments(self, command_string):
        """
        Parse the given command string "command_string" into arguments
        and check if all required arguments are present. Use the "_convert_type"
        method to ensure each argument is stored with the correct type in the "arguments"
        dictionary. Begin by splitting "command_string" using the space delimiter
        after the script name to separate the arguments.
        Handle both short ('-') and long ('--') argument prefixes,
        and for each argument, split or assign the value accordingly.
        Use "self._convert_type" to convert argument values to their correct
        types as specified in "self.types". Then, create a set "missing_args"
        of required arguments from "self.required" that are not in
        the parsed "self.arguments". Return a tuple where the first element
        is "True" if all required arguments are present, otherwise "False",
        and the second element is "None" if no arguments are missing,
        otherwise the set "missing_args" containing names of missing required arguments.
        :param command_string: str, command line argument string, formatted
                like "python script.py --arg1=value1 -arg2 value2 --option1 -option2"
        :return tuple: (True, None) if parsing is successful, (False, missing_args) if parsing fails,

        """
```

46. In a few words, what is the task this function aims to implement? *

47. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

48. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

49. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Demographics Information

50. What is your highest education qualification? *

Une seule réponse possible.

- ◯ Ph.D.
- ◯ Master's Degree
- ◯ Bachelor Degree
- ◯ Autre : _____

51. What is your field of work? (Engineer, Researcher, Developer, etc.) *

_____

52. How many years of development experience do you have? *

Une seule réponse possible.

- ◯ Less than 5
- ◯ Between 5 and 10
- ◯ More than 10

53. Thank you for completing the survey! Any feedback you would like to provide?

_____

_____

_____

_____

_____